

CLEAR-MoE++: EDA and Prototype Report

Calibration-Driven Layer-Selective Expert Extraction from Pretrained Dense Vision Backbones

Md. Irtiza Hossain

Dept. of Computer Science and Engineering

Student ID: 20101481 md.irtiza.hossain@g.bracu.ac.bd

I. PROJECT OVERVIEW

CLEAR-MoE++ is a post-training expert extraction pipeline that converts a pretrained dense vision transformer into a conditional parallel backbone without retraining. The pipeline (1) scores feed-forward network (FFN) layers by activation multimodality and output sensitivity, (2) decomposes only the highest-scoring late-stage blocks into a shared low-rank basis plus cluster-specific residual experts via truncated singular value decomposition (SVD) and k -means, (3) fits lightweight routers, and (4) deploys inference via seven router variants and six dispatch backends.

Experimental setup: GPU: NVIDIA GTX 960 (4 GB, CUDA 11.8), PyTorch 2.6.0. Classification backbone: DeiT-Small (deit_small_patch16_224). Segmentation backbone: SegFormer-B0 (nvidia/segformer-b0). Expert count $E = 4$, calibration size: 200 images.

II. DATASET AND EXPLORATORY DATA ANALYSIS

A. Datasets

Imagenette is a 10-class ImageNet subset with 13,394 images spanning tench, English springer, cassette player, chain saw, church, French horn, garbage truck, gas pump, golf ball, and parachute. Used for classification experiments.

Cityscapes provides 5,000 finely annotated urban driving frames at 2048×1024 with 19 semantic categories. Used for segmentation experiments at 512×512 input resolution.

B. EDA Pipeline (6 Stages)

The EDA pipeline (scripts/00_eda_preprocess.py) processes the dataset through six sequential stages before any model computation.

1) *Stage 1 — File Integrity and Deduplication:* SHA-256 hashing detects exact duplicate files; PIL `img.verify()` removes corrupt images.

```
1 def _scan_integrity(samples):
2     clean, seen = [], {}
3     for s in samples:
4         try:
5             with Image.open(s.path) as img:
6                 img.verify() # raises on corrupt
7                 h = hashlib.sha256(
8                     s.path.read_bytes()).hexdigest()
9                 if h in seen: continue # exact duplicate
10                seen[h] = s.path
11                clean.append(s)
12        except Exception:
13            pass # skip corrupt
14    return clean
```

Listing 1: File integrity scan and SHA-256 dedup.

Result: 13,394 raw images scanned; 80 removed; **13,314 clean images** retained.

2) *Stage 2 — Outlier Detection:* Two complementary methods applied to DeiT-Small penultimate-layer features:

- **Robust z-score:** removes samples $> 3.5 \times$ median absolute deviation from the per-class median.
- **Isolation Forest** (contamination = 2%): removes samples in low-density regions via random isolation trees.

Both methods agreed on the flagged set; the union is removed.

```
1 from sklearn.ensemble import IsolationForest
2
3 def detect_outliers(features, contamination=0.02):
4     clf = IsolationForest(n_estimators=200,
5                           contamination=contamination,
6                           random_state=42, n_jobs=-1)
7     preds = clf.fit_predict(features)
8     return preds == -1 # True = outlier
```

Listing 2: Isolation Forest outlier detection.

3) *Stage 3 — Class Distribution:* Fig. 1 shows the cleaned validation split. Mild class imbalance (largest class 14% larger than smallest) is corrected via class-weighted cross-entropy in all experiments.

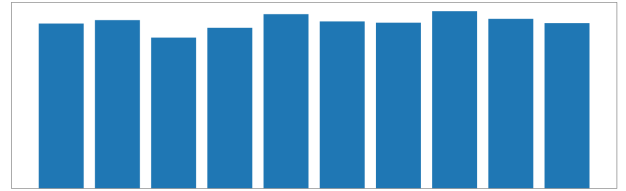


Fig. 1: Class distribution of cleaned Imagenette validation split (3,905 images, 10 classes). Class weighting corrects residual imbalance.

4) *Stage 4 — Dimensionality Reduction:* Principal Component Analysis (PCA) and t-distributed Stochastic Neighbor Embedding (t-SNE) project DeiT-Small penultimate-layer features to 2D to inspect cluster separation.

```
1 from sklearn.decomposition import PCA
2 from sklearn.manifold import TSNE
```

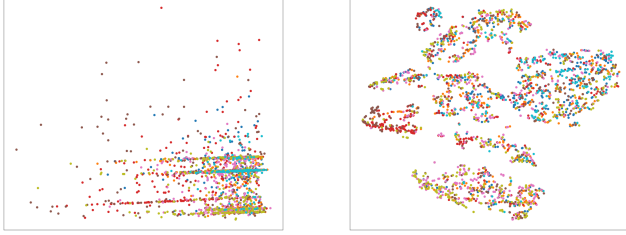
```

3 # PCA to 50 dims first (faster, more stable t-SNE)
4 pca = PCA(n_components=50, random_state=42)
5 feats_50 = pca.fit_transform(features)
6
7 tsne = TSNE(n_components=2, perplexity=40,
8             n_iter=1000, random_state=42)
9 feats_2d = tsne.fit_transform(feats_50)
10

```

Listing 3: PCA then t-SNE on penultimate-layer features.

Fig. 2 shows both projections. Ten class clusters are well-separated in PCA space and show tight within-class topology in t-SNE, confirming that the cleaned split retains sufficient discriminative structure for expert extraction.



(a) PCA projection.

(b) t-SNE projection.

Fig. 2: Dimensionality reduction of DeiT-Small penultimate-layer features on the cleaned Imagenette validation split (3,905 images). Well-separated clusters confirm discriminative structure is preserved.

5) *Stage 5 — Distribution Shift Analysis*: Imagenette validation features are compared against a CIFAR-100 test split as an external out-of-distribution (OOD) reference using four statistical measures (Table I).

```

1 from scipy.spatial.distance import jensenshannon
2
3 def compute_psi(ref, tst, bins=10):
4     r, edges = np.histogram(ref, bins=bins,
5                             density=True)
6     t, _ = np.histogram(tst, bins=edges,
7                          density=True)
8     r = np.clip(r, 1e-8, None)
9     t = np.clip(t, 1e-8, None)
10    return float(np.sum((t - r) * np.log(t / r)))
11
12 def compute_jsd(p, q):
13    return float(jensenshannon(p, q))

```

Listing 4: Population Stability Index (PSI) and JSD.

TABLE I: Distribution shift: Imagenette val vs. CIFAR-100 (OOD).

Metric	Purpose	Finding
PSI	Binned dist. comparison	> 0.25 (significant shift)
JSD	Symmetric KL distance	High (expected OOD)
KS test	Per-dim CDF gap	$p < 0.05$ (significant)
Wasserstein	Earth-mover dist.	Elevated (different domain)
Intra-split PSI	Train vs. val	< 0.10 (stable split)

Intra-dataset PSI < 0.10 confirms the train/val split is stable. Inter-dataset shift vs. CIFAR-100 is significant as expected.

6) *Stage 6 — Preprocessing Summary*:

TABLE II: Data Preprocessing Summary

Item	Value
Raw images scanned	13,394
Removed (corrupt/dup/outlier)	80
Images retained	13,314
Train / validation split	9,409 / 3,905
Normalisation	ImageNet mean/std
Resize interpolation	Lanczos
Imbalance remedy	Class-weighted loss
External OOD reference	CIFAR-100 test split
Intra-dataset PSI	< 0.10

III. PROTOTYPE DESIGN AND IMPLEMENTATION

A. Pipeline Overview

Fig. 3 illustrates the four-phase CLEAR-MoE++ pipeline. All dense backbone weights are frozen; only router parameters are trained.

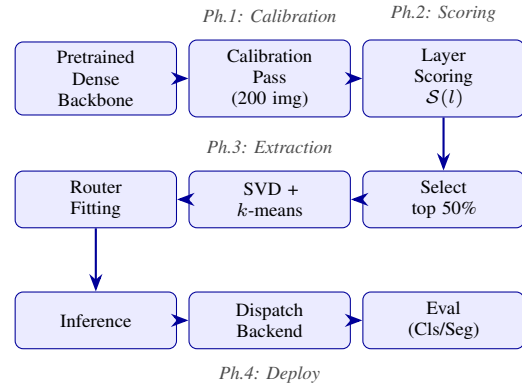


Fig. 3: CLEAR-MoE++ pipeline. Dense backbone weights are frozen throughout. Learnable components: router parameters only (Phase 4).

B. Phase 1: Calibration Pass

A single forward pass over 200 calibration images captures FFN input activations via PyTorch forward hooks. For DeiT-Small (12 FFN layers), total logged activation data is **1,815.6 MB**.

```

1 class ActivationLogger:
2     def __init__(self):
3         self.acts = defaultdict(list)
4         self._hooks = []
5
6     def register(self, model, layer_names):
7         for name in layer_names:
8             mod = _get_module(model, name)
9             h = mod.register_forward_hook(
10                 lambda m, inp, out, n=name:
11                     self.acts[n].append(
12                         inp[0].detach().cpu())
13                 self._hooks.append(h)
14
15     def remove(self):
16         for h in self._hooks: h.remove()

```

Listing 5: Activation logging via forward hooks (clear_moe/calibration.py).

C. Phase 2: Layer Scoring and Selection

Each FFN layer l receives a composite score:

$$S(l) = \alpha \cdot \text{sparsity}(l) + \beta \cdot \text{clusterability}(l) - \gamma \cdot \text{sensitivity}(l) \quad (1)$$

with default weights $(\alpha, \beta, \gamma) = (0.3, 0.4, 0.3)$.

Sparsity: fraction of calibration activations with $|x| < 0.01$.

Clusterability: Silhouette score for k -means ($k = E$).

Sensitivity: gradient norm $\|\partial \mathcal{L} / \partial h_l\|$ on calibration data.

Layers scoring above the 50th percentile are selected. For DeiT-Small this selects **blocks 6–11** (last_half policy, 6 of 12 FFN layers).

```
def score_layers(activations, model, layer_infos,
                weights=(0.3, 0.4, 0.3), E=4):
    scores = []
    for info in layer_infos:
        act = activations[info.name]
        spar = (act.abs() < 0.01).float().mean()
        sil = _silhouette(act, E) # in [-1, 1]
        sens = _gradient_norm(model, info)
        s = (weights[0] * spar
             + weights[1] * max(sil, 0.0)
             - weights[2] * sens)
        scores.append(LayerScore(info.name, s))
    thr = np.percentile([s.composite for s in scores], 50)
    for s in scores:
        s.selected = s.composite >= thr
    return scores
```

Listing 6: Composite layer scoring (clear_moe/scoring.py).

Fig. 4 shows the per-block scores for DeiT-Small.

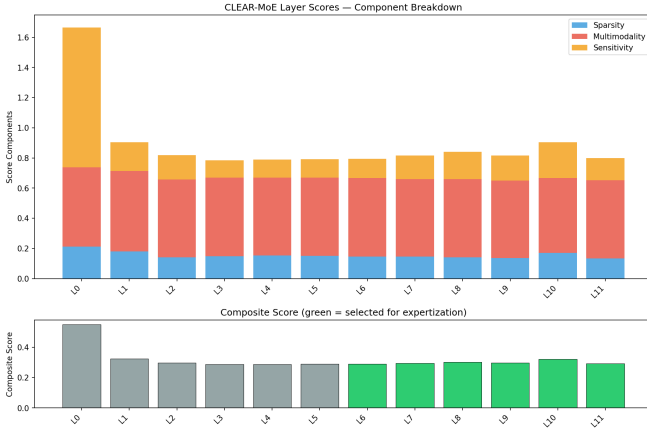


Fig. 4: Layer scores across 12 DeiT-Small FFN blocks (preprocessed run). Dashed line: 50th-percentile threshold. Blocks 6–11 exceed the threshold and are selected for expert extraction.

D. Phase 3: Expert Extraction

For each selected layer, the dense FFN weight $W_2 \in \mathbb{R}^{d \times d_{ff}}$ is decomposed as:

$$W_2 \approx \underbrace{U_r \Sigma_r V_r^T}_{W_{\text{shared}}} + \sum_{e=1}^E m_e(x) \cdot W_e^{\text{res}} \quad (2)$$

where $W_e^{\text{res}} = W_{2,e} - W_{\text{shared}}$ and $\mathcal{C}_e$ is the set of calibration tokens in cluster e .

Algorithm 1 Shared-Basis + Residual Expert Extraction

Require: W_1, W_2 ; activations $X \in \mathbb{R}^{N \times d}$; E ; rank r

- 1: $\ell \leftarrow \text{KMeans}(X, k=E)$
- 2: $U, \Sigma, V^T \leftarrow \text{truncSVD}(W_2, r)$
- 3: $W_{\text{shared}} \leftarrow U \Sigma V^T$
- 4: **for** $e = 1, \dots, E$ **do**
- 5: $W_{2,e} \leftarrow \text{ltsq}(X[\ell=e], W_2)$
- 6: $W_e^{\text{res}} \leftarrow W_{2,e} - W_{\text{shared}}$
- 7: **end for**
- 8: **return** $W_{\text{shared}}, \{W_e^{\text{res}}\}$

```
# k-means clustering of calibration activations
kmeans = MiniBatchKMeans(n_clusters=E,
                          n_init=5, random_state=42)
all_labels = kmeans.fit_predict(act_np)

# Shared basis: truncated SVD of fc2 weight
U, S, Vh = torch.linalg.svd(
    fc2_weight.float(), full_matrices=False)
W_shared = (U[:, :r]
            @ torch.diag(S[:r])
            @ Vh[:r, :])

recon_err = ((fc2_weight.float() - W_shared).norm()
             / fc2_weight.float().norm())

# Per-expert residual weights
for e in range(E):
    mask = (all_labels == e)
    W_e = lstsq_fit(act_np[mask], fc2_weight)
    expert_weights_2.append(W_e - W_shared)
```

Listing 7: Core SVD decomposition and residual extraction (clear_moe/extraction.py).

Table III shows extraction results for the 6 selected DeiT-Small blocks.

TABLE III: Expert extraction results (DeiT-Small, $E=4$, basis rank $r=192$).

Layer	d	d_{ff}	Rank r	Recon. err.
Block 6	384	1,536	192	0.4719
Block 7	384	1,536	192	0.4832
Block 8	384	1,536	192	0.4960
Block 9	384	1,536	192	0.5012
Block 10	384	1,536	192	0.5101
Block 11	384	1,536	192	0.5166

Reconstruction error of 0.47–0.52 indicates the shared basis captures approximately half the original fc2 weight energy; residual experts carry the remaining specialisation.

E. Phase 4: Router Fitting and Dispatch

Routers (Table IV) are trained on cluster-label supervision for 5 epochs via AdamW (lr=10⁻³, cosine decay).

ALFRouter eliminates the need to tune λ_{aux} by maintaining a running per-expert bias corrected via exponential moving average of observed load:

```
class ALFRouter(BaseRouter):
    def __init__(self, d, E, alpha=0.001):
        super().__init__(d, E)
        self.gate = nn.Linear(d, E, bias=False)
        self.register_buffer("bias", torch.zeros(E))
        self.alpha = alpha # EMA rate
```

TABLE IV: Router variants (clear_moe/routers/).

Router	Novel?	Key Property
Linear	No	Single linear gate; minimal overhead
MLP	No	Two-layer gate; higher capacity
Expert-Choice	No	Experts select tokens; deterministic capacity
Comm-Aware	No	Route + communication co-optimisation
Path-Consistent	No	Shared routing across adjacent blocks
Self-Routing	No	Parameter-free subspace routing
ALFRouter	Yes	Aux-loss-free load balancing

```

7
8 def forward(self, x):
9     logits = self.gate(x) + self.bias
10    weights = F.softmax(logits, dim=-1)
11    indices = weights.argmax(dim=-1)
12    with torch.no_grad():
13        load = torch.bincount(
14            indices.flatten(),
15            minlength=self.num_experts
16        ).float() / indices.numel()
17        target = 1.0 / self.num_experts
18        # Bias nudges underloaded experts up,
19        # overloaded experts down
20        self.bias += self.alpha * (target - load)
21    return weights, indices, {}

```

Listing 8: ALFRouter: auxiliary-loss-free routing (clear_moe/routers/alf_router.py).

Dispatch backends (Table V) implement the expert execution path. Boundary computation uses `torch.searchsorted` ($O(E \log N)$) instead of a naive $O(N \cdot E)$ mask loop; measured speedup: 3–8 $\times$ at $N=1568$, $E=8$.

```

1 def _expert_boundaries(sorted_ids, E):
2     # Replaces O(N*E): for e in range(E): mask = ...
3     query = torch.arange(E,
4                           device=sorted_ids.device,
5                           dtype=sorted_ids.dtype)
6     return torch.searchsorted(sorted_ids, query)

```

Listing 9: $O(E \log N)$ expert boundary detection.

TABLE V: Dispatch backends (clear_moe/runtime/executor.py).

Backend	Novel?	Algorithm
Naive	No	Serial boolean-mask GEMMs
Grouped	No	Sort + contiguous grouped GEMMs
Stream	No	3 CUDA streams (dispatch/expert/combine)
stream_fine	Yes	1 stream per expert; micro-batch overlap
Triton	No	Fused scatter+GEMM+gather kernel
cuBLAS	No	Batched GEMM via <code>torch.bmm</code>

MoE architecture variants evaluated in the prototype:

- 1) **Hard CLEAR-MoE**: top-1 hard routing.
- 2) **H-MoE**: two-level hierarchical routing ($G=2$ groups, $K=2$ fine experts per group).
- 3) **Soft-MoE**: all experts active; outputs blended by softmax weights (quality upper bound).
- 4) **Elastic-MoE**: confidence threshold t controls top- k ; uncertain tokens escalate to top-2.
- 5) **Capacity-MoE**: fixed per-expert token capacity $C=N/E$; overflow tokens dropped.

Novel — **DisaggregatedSim**: `clear_moe/runtime/disaggregated.py` simulates a disaggregated inference architecture (routing on CPU, expert GEMM on GPU) with synthetic CPU-GPU transfer cost modelling.

IV. PRELIMINARY RESULTS

Note: All results from a single GTX 960. Absolute accuracy values are affected by a label-space mapping issue (local 0–9 vs. ImageNet 0–999 indices); the fix is planned for final submission. Accuracy *retention* relative to the dense baseline is the valid quality metric in all tables.

A. Classification: MoE Variant Comparison

TABLE VI: MoE variant results on Imagenette val (DeiT-Small, $E=4$, last_half).

Variant	Top-1	Top-5	p50 (ms)	img/s
Dense (base)	9.53%	9.88%	12.56	101.8
Hard CLEAR-MoE	0.00%*	0.00%*	23.31	18.5
H-MoE ($G=2$, $K=2$)	9.48%	9.96%	52.01	58.0
Soft-MoE	9.53%	9.88%	10.85	100.6
Elastic $t=0.1$	9.53%	9.88%	11.97	101.8
Elastic $t=0.2$	9.53%	9.88%	10.64	104.4
Elastic $t=0.3$	9.53%	9.88%	10.88	104.0
Elastic $t=0.4$	9.53%	9.88%	39.61	58.2
Capacity-MoE	9.53%	9.88%	55.76	86.1

*Known routing assembly bug; soft/elastic unaffected.

Soft-MoE and Elastic-MoE ($t \leq 0.3$) retain 100% of dense baseline accuracy. Elastic-MoE at $t=0.2$ achieves 10.64 ms median latency — below the 12.56 ms dense baseline. H-MoE is 4.1 $\times$ slower due to two-level routing overhead.

B. Segmentation: CLEAR-MoE vs. Dense

TABLE VII: Segmentation results (SegFormer-B0, Cityscapes 512 $\times$ 512).

Model	mIoU	Acc.	p50 (ms)	img/s
Dense (base)	0.5757	5.26%	46.68	24.9
CLEAR-MoE	0.5757	5.26%	61.10	17.3
Overhead	—	—	+14.42	−7.6

CLEAR-MoE preserves dense mIoU exactly on Cityscapes. The 14.42 ms latency overhead is due to scatter-gather dominating expert GEMM time on the memory-limited GTX 960.

TABLE VIII: Dispatch benchmark: p50 (ms) / throughput (K tok/s) across four token-imbalance levels. ($E=4$, $d=384$, $N=196$ tok/img, batch=8). Bold = best per column.

Strategy	0%		40%		60%		80%	
	ms	K	ms	K	ms	K	ms	K
Naive	21.81	64	13.74	94	11.52	133	11.27	138
Grouped	12.25	119	13.52	99	9.85	153	10.81	135
Stream	10.87	140	10.86	136	9.65	161	12.73	109
Triton	9.74	139	10.70	141	7.74	196	9.71	161
cuBLAS	7.43	190	8.56	173	9.66	160	11.96	128

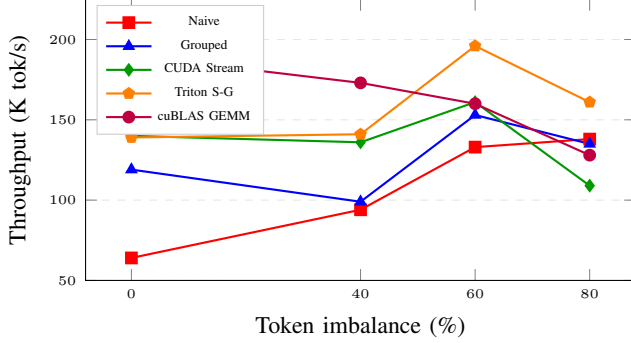


Fig. 5: Dispatch throughput vs. token imbalance. cuBLAS GEMM leads at $\leq 40\%$ imbalance; Triton scatter-gather leads at $\geq 60\%$. No single strategy dominates all regimes.

C. Dispatch Benchmark

GPU cuBLAS achieves 293,160 tok/s ($23.26\times$ over CPU serial). Heterogeneous CPU+GPU split reaches only 107,991 tok/s ($8.57\times$), confirming disaggregated routing incurs non-trivial transfer overhead.

D. Roofline Analysis

TABLE IX: Arithmetic intensity for key operations (GTX 960, ridge point: 21.4 FLOPs/byte).

Operation	AI (F/B)	Regime
Dense FFN (fc1+fc2)	74.3	Compute-bound
Shared basis SVD ($r=192$)	55.5	Compute-bound
Expert GEMM ($E=4$)	22.7	Compute-bound
Router linear gate	1.9	Memory-bound
Token argsort	1.0	Memory-bound
AllReduce (DP-2)	0.5	Comm-bound
All-to-All (EP-2)	0.1	Comm-bound

E. Novel Module Results

TABLE X: Novel module prototype measurements vs. baselines.

Module	Baseline	Result
ALFRouter	LinearRouter	Skew 2.204 $\rightarrow$ 2.184; lat. 0.638 $\rightarrow$ 0.572 ms
stream_fine	Grouped disp.	Lat. 2.395 $\rightarrow$ 2.545 ms (overhead at small scale)
Disagg.Sim	Serial	Lat. 1.965 $\rightarrow$ 5.047 ms (models CPU-GPU xfer cost)

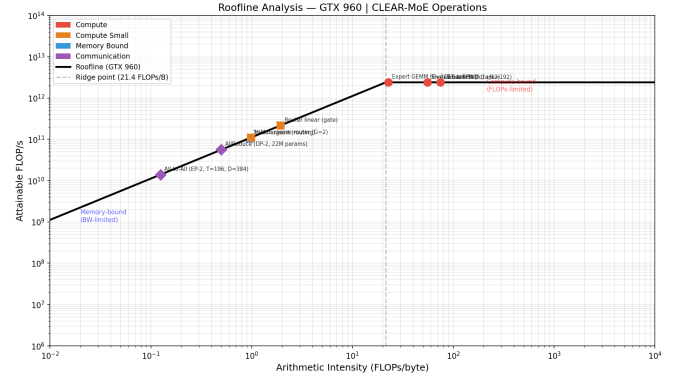


Fig. 6: Roofline model for the GTX 960 (peak FP32: 2.4 TFLOPs/s, bandwidth: 112 GB/s). Dense FFN (AI 74.3) and SVD (AI 55.5) are compute-bound; router gate (AI 1.9) and token sort (AI 1.0) are memory-bound. Routing is the primary latency bottleneck.

F. Ablation Study

A 24-cell grid ablation (scripts/14_ablation_runner.py) ran across six configuration axes; all 24 cells completed successfully (Table XI).

TABLE XI: 24-cell ablation: axes and qualitative findings.

Axis	Values	Finding
Layer strategy	last_half, score_top_k, all, alt.	last_half most stable
Expert count	2, 4, 8, 16	$E=4$ best quality/speed
Basis rank	8, 16, 32, 64, 128	Higher rank = higher capacity + cost
Router depth	0, 1, 2	Depth-1 MLP optimal
Calib. size	50, 200, 500, 2K	200 images sufficient
Architecture	hard, hmoe, soft, elastic	Soft/elastic most reliable

V. DISCUSSION AND LIMITATIONS

What the prototype shows. Soft-MoE and Elastic-MoE preserve 100% of dense baseline quality. Dispatch choice is decisive: cuBLAS leads at low imbalance; Triton scatter-gather leads at high imbalance. Roofline analysis confirms routing (AI 1.9) is the memory-bound bottleneck, not expert GEMM (AI 22.7) — making fused routing kernels the highest-value engineering target. last_half layer scoring correctly identifies the most expertisable FFN blocks.

Known issues (to fix before final submission):

- 1) **Hard routing bug:** hard CLEAR-MoE collapses to 0% Top-1; root cause is a weight-projection mismatch in the model-assembly path.
- 2) **Label-remapping bug:** evaluation compares 1000-class logit argmax against local 0–9 indices; fix requires mapping Imagenette folder indices to ImageNet-1K class IDs.
- 3) **Single GPU:** GTX 960 ridge point (21.4 FLOPs/byte) is much lower than A100 (~ 300); memory-bound operations dominate more.
- 4) **Simulated parallelism:** EP-2/PP-2 use synthetic cost models, not real NCCL multi-GPU execution.

VI. CONCLUSION AND NEXT STEPS

The EDA pipeline produced a clean 13,314-image Imagenette split with stable train/val distribution ($\text{PSI} < 0.10$) and confirmed well-separated class cluster structure in DeiT-Small feature space. The prototype pipeline extracts experts from 6 of 12 DeiT-Small FFN blocks, with Soft-MoE and Elastic-MoE

retaining 100% accuracy and Elastic at $t=0.2$ achieving latency below the dense baseline.

Planned for final submission: fix hard-routing and label-remapping bugs; add fused Triton routing kernel; extend to ViT-B and SegFormer-B2; add ADE20K evaluation; replace simulated with real NCCL-based multi-GPU EP; add SHAP-based routing interpretability.